

TrueSight Autopilot — Infrastructure Diagnosis & Upgrade Proposal

Date: June 6, 2026

Prepared by: Sophia (TrueSight DAO Autopilot)

For: Gary Teh (Governor)

1. Current State — Full Diagnosis

Instance Identity

- **Account:** Explorya (440626669078)
- **Instance ID:** i-02c699d3d7efbdc82
- **Instance Type:** t3.small
- **Public IP:** 52.200.38.206
- **Tag:** truesight-autopilot
- **AMI:** Ubuntu 22.04 LTS
- **Launched:** May 3, 2026 (recently recreated June 3)

CPU

- **vCPUs:** 2 (1 core × 2 threads)
- **Architecture:** Intel Xeon Platinum 8259CL @ 2.5 GHz
- **Burstable:** Yes (t3 credits — can burst to full 2 vCPUs when credits available)
- **Load average:** 0.32 / 0.21 / 0.13 — very low, CPU is not the bottleneck

Memory

- **Total:** 2,048 MB (2 GB)
- **Used (steady state):** ~900 MB
- **Available:** ~822 MB
- **Swap:** None
- **Assessment:** 2 GB is tight. During deploys, pip installs + git clones + process restarts spike memory. The deploy failure (exit -15 / SIGTERM) was almost certainly an OOM kill during the `dao\_client` pip install phase.

Disk

- **Volume:** gp3, 20 GB (EBS)
- **Used:** 6.8 GB (35%)
- **Available:** 13 GB
- **Assessment:** 20 GB is actually adequate for now. The earlier 80% figure was from a different box. No immediate disk pressure.

Network

- **Baseline bandwidth:** 128 Mbps (t3.small baseline)
- **Peak burst:** Up to 5 Gbps
- **Assessment:** Fine for current load. The autopilot is API-driven, not bandwidth-intensive.

Running Services

- **uvicorn** (main autopilot API, port 8001, 2 workers)
- **Telegram adapter** (multiprocessing, 2 child processes)
- **dao_protocol** (FastAPI, port 8010) — separate service on same box
- **Monit** (port 2812) — process monitoring

Deploy Failure Root Cause

The deploy script runs from your local machine (Mac) and SSHes into the autopilot box. Phase 1 (git pull) succeeded. Phase 2 failed with exit code -15 (SIGTERM) during `pip install truesight-dao-client@git+...`. The pip install process was killed, likely by the kernel OOM killer when memory spiked above the 2 GB ceiling.

2. Proposed Target State

Recommended Instance: t3.medium

Resource	Current (t3.small)	Proposed (t3.medium)	Improvement
vCPU	2	2	Same (adequate)
RAM	2 GB	4 GB	2x — eliminates OOM during deploys
Network baseline	128 Mbps	256 Mbps	2x — better API responsiveness
EBS baseline IOPS	1,000	2,000	2x — faster pip installs, git ops
Disk	20 GB	20 GB	Keep (adequate)

Why t3.medium (not t3.large)?

- t3.large has 8 GB RAM but costs ~\$69/month vs ~\$35/month for t3.medium
- 4 GB is the sweet spot — enough headroom for deploys without over-provisioning
- CPU load is consistently low — 2 vCPUs is fine
- If we need more later, we can resize again (t3 instances support reboot migration)

Additional Recommendations

1. **Add 2 GB swap** — safety net for transient spikes, costs nothing
2. **Enable detailed CloudWatch monitoring** — so we can see memory metrics historically
3. **Consider a reserved instance** — 1-year term drops t3.medium from ~\$35 to ~\$22/month

Cost Comparison

| Option | On-Demand (monthly) | 1-Year Reserved |

|---|---|---|

| **Current:** t3.small | ~\$21 | ~\$13 |

| **Proposed:** t3.medium | ~\$35 | ~\$22 |

| **Delta** | **+\$14/month** | **+\$9/month** |

3. Upgrade Procedure

1. **Stop the autopilot services** (systemctl stop truesight-autopilot truesight-autopilot-telegram)
2. **Change instance type** in AWS Console: EC2 → Instances → i-02c699d3d7efbdc82 → Actions → Instance Settings → Change Instance Type → t3.medium
3. **Add swap:** ``sudo fallocate -l 2G /swapfile && sudo chmod 600 /swapfile && sudo mkswap /swapfile && sudo swapon /swapfile``
4. **Make swap permanent:** add to /etc/fstab
5. **Restart services** (systemctl start truesight-autopilot truesight-autopilot-telegram)
6. **Redeploy** to verify the fix

Total downtime: ~2-3 minutes.

4. Fleet Context

For reference, here's how the autopilot box compares to the rest of the fleet:

Host	Type	vCPU	RAM	Disk	Role
---	---	---	---	---	---
Autopilot (YOU)	t3.small	2	2 GB	20 GB	Autopilot + dao_protocol
seni_ror	t2.small	1	2 GB	24 GB	Edgar Rails
seni_sk	t2.small	1	2 GB	8 GB	Edgar Sidekiq
seni_sql	t2.small	1	2 GB	—	Edgar PostgreSQL
krake_ror	t2.micro	1	1 GB	—	Krake Rails
krake_data	t3.medium	2	4 GB	—	Krake data processing
krake_nginx	t2.micro	1	1 GB	—	Nginx reverse proxy

The autopilot box is the only one running TWO services (autopilot + dao_protocol), making the 2 GB RAM constraint more acute than it appears.

End of report.